Solution: Longest Nice Substring

Problem Statement

Given a string `str`, return the longest `nice` substring of a given string.

A substring is considered nice if for every lowercase letter in the substring, its uppercase counterpart is also present, and vice versa.

If no such string exists, return an empty string.

Examples

1. **Example 1**:
    - Input: `"BbCcXxY"`
    - Expected Output: `"BbCcXx"`
    - Justification: Here, `"BbCcXx"` is the longest substring where each letter's uppercase and lowercase forms are present.
2. **Example 2**:
    - **Input**: `"aZAbcD"`
    - **Expected Output**: `""`
    - **Justification**: There is no contiguous substring where each character exists in both its uppercase and lowercase forms.
3. **Example 3**:
    - Input: `"qQwWeErR"`
    - Expected Output: `"qQwWeErR"`
    - Justification: The entire string is the longest nice substring since every letter exists in both uppercase and lowercase forms.

Solution

To solve this problem, you can use a straightforward approach. First, check if the entire string is 'nice', which means it has each letter in both uppercase and lowercase. If it's not, then you start breaking the string into smaller parts. You do this by dividing the string around each character, creating two new segments each time: one on the left and one on the right of that character.

This splitting helps you explore every possible combination to find a 'nice' substring. For each of these smaller strings, you apply the same 'nice' criteria. If a substring qualifies, compare its length with your current longest 'nice' substring and keep the longer one. Continue this process of dividing and checking each part of the string.

Through this method, you ensure that no potential 'nice' substring is missed.

Step-by-Step Algorithm:

1. **Base Condition**:
    - If the string is empty or has a length of 1, it cannot be a nice substring. Return an empty string in such cases.
2. **Check for Nice Substring**:
    - Scan the entire string to check if it is a nice substring. This involves checking if, for every lowercase letter, its uppercase counterpart is present and vice versa.
3. **Divide if Not Nice**:
    - If the current string is not a nice substring, divide the problem. This involves breaking the string into two halves at each character and recursively applying the algorithm on these halves.
    - For each character in the string:
        - Exclude the character and divide the string into two parts: left and right of the excluded character.
        - Recursively call the function on these two parts to find the longest nice substring in each part.
4. **Conquer**:
    - From the recursive calls, two substrings are returned, one from the left part and one from the right. Compare these substrings based on their lengths.
    - Choose the longer of the two substrings as the longest nice substring for the current recursive call.
5. **Combine and Return**:
    - Continue combining the results of the recursive calls as the execution stack unwinds.
    - Eventually, return the longest nice substring found for the entire string.

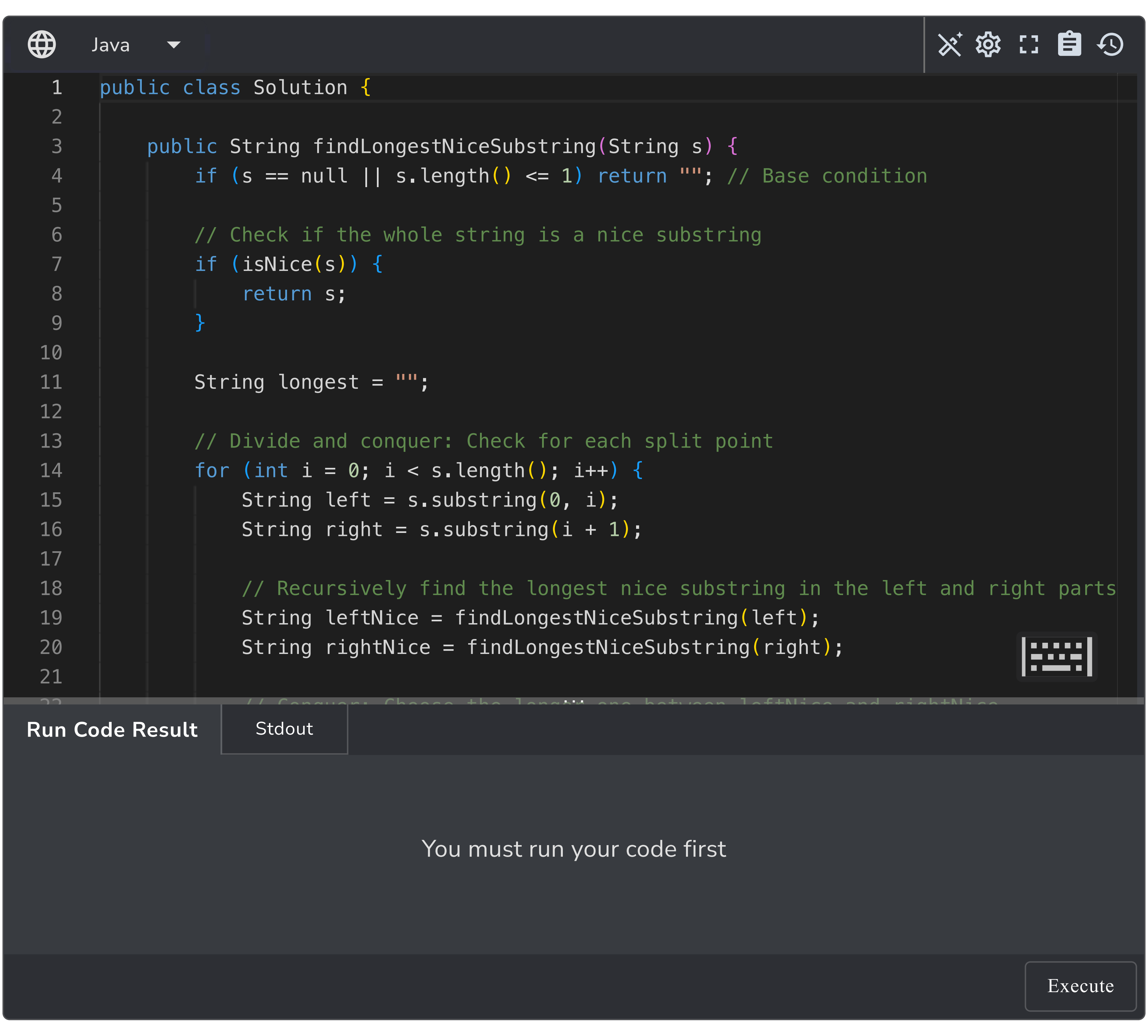
Algorithm walkthrough

Consider input "BbCcXxy".

1. **Initial Check:**
    - Start by checking if the entire string `"BbCcXxY"` is nice. It's not, because 'Y' does not have a lowercase counterpart.
2. **First Division:**
    - Divide the string around each character and recursively process the substrings.
    - First division is around 'B': left substring `""`, right substring `"bCcXxY"`.
    - Continue dividing the right substring.
3. **Divide around 'b'**:
    - left substring `"B"`, right substring `"CcXxY"`.
    - `"B"` is not nice. Continue with `"CcXxY"`.
4. **Divide around 'C'**:
    - left substring `"Bb"`, right substring `"cXxY"`.
    - `"Bb"` is nice, but keep checking for a longer nice substring.
    - Continue with `"cXxY"`.
5. **Divide around 'c'**:
    - left substring `"BbC"`, right substring `"XxY"`.
    - `"BbC"` is not nice. Continue with `"XxY"`.
6. **Divide around 'X'**:
    - left substring `"BbCc"`, right substring `"xY"`.
    - `"BbCc"` is nice, longer than `"Bb"` Continue with `"xY"`.
7. **Divide around 'x'**:
    - left substring `"BbCcX"`, right substring `"Y"`.
    - `"BbCcX"` is not nice. Continue with `"x"`.
8. **Divide around 'y'**:
    - left substring `"BbCcXx"`, right substring `""`.
    - `"BbCcXx"` is nice, longer than `"BbCc".
9. **Final Output:**
    - The longest nice substring found is `"BbCcXx"`.

Code

Here is the code for this algorithm:



```java
public class Solution {

    public String findLongestNiceSubstring(String s) {
        if (s == null || s.length() <= 1) return ""; // Base condition

        // Check if the whole string is a nice substring
        if (isNice(s)) {
            return s;
        }

        String longest = "";

        // Divide and conquer: Check for each split point
        for (int i = 0; i < s.length(); i++) {
            String left = s.substring(0, i);
            String right = s.substring(i + 1);

            // Recursively find the longest nice substring in the left and right parts
            String leftNice = findLongestNiceSubstring(left);
            String rightNice = findLongestNiceSubstring(right);

```

You must run your code first

Time and Space Complexity Analysis

Time Complexity: The time complexity is O(n^3), where n is the length of the input string. This is due to the nested structure of the algorithm, involving iteration over each character, recursive calls, and checking if each substring is nice.

Space Complexity: The space complexity is O(n), primarily due to the recursion stack and additional space for substring operations, with n being the depth of recursion equal to the string length.

← Previous
Longest Nice Substring (Easy)
Next →
Sort List (Medium)
☐ Mark as Completed